Capture the Flag 8: Enroll a New Student

Goal: Extend your multi-screen Student Directory by implementing student enrollment using form fields, controllers, validation, submission, and local data persistence while preserving your existing navigation, student details, and edit modal.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does.
- Understand syntax.
- Explain an error (not fix your error).
- Clarify documentation.

You may NOT ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

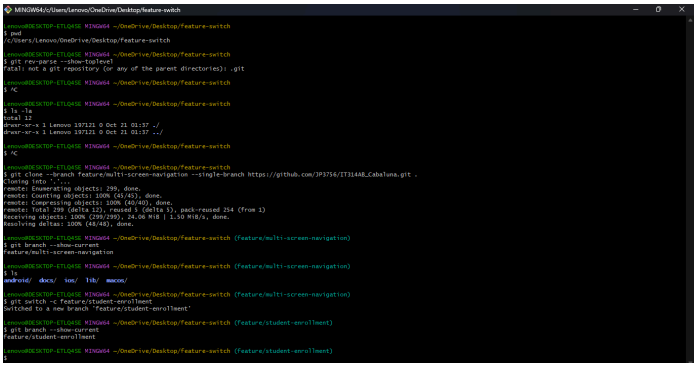
Objective: Create a new feature branch before implementing student enrollment.

Create:

`feature/student-enrollment`

Screenshot:

- Terminal showing your current branch.



🏴 Flag 1 — Build the Add Student Form

Objective: Transform your existing Add Student Screen into a functional enrollment form.

Add the following input fields:

- Student ID
- Full Name
- Course
- Year Level
- Email Address

Research how Flutter uses:

- `TextField` or `TextFormField`
- `TextEditingController`
- Form layouts and input fields

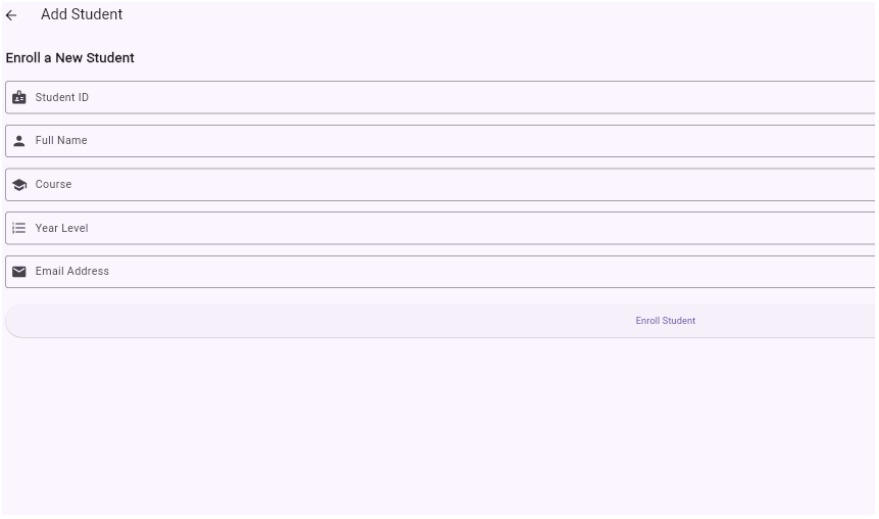
Test

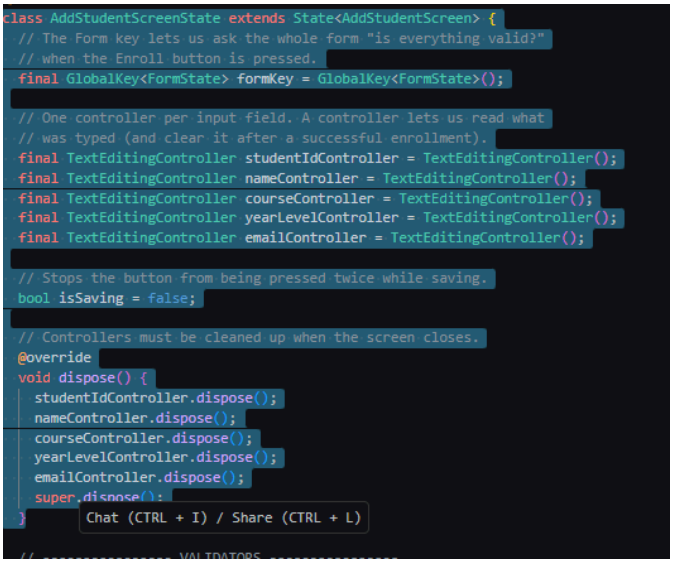
The Add Student Screen should display all required fields and allow administrators to enter student information.

The screen must remain accessible through your existing Student List navigation.

Screenshot:

- Add Student Screen with all input fields.
- Controllers declared in your code.
- Application displaying the enrollment form.





🏴 Flag 2 — Validate Student Information

Objective: Prevent incomplete, invalid, or duplicate student information from being submitted.

Implement form validation with the following requirements:

- All required fields must be filled in.
- Student ID must not duplicate an existing student's ID.
- Email Address must follow a valid email format.
- Year Level must contain an appropriate year level value.

Research how Flutter handles form validation and displays validation messages.

Test

1. Submit the form with all fields empty.
2. Enter an invalid email address.
3. Enter a Student ID that already exists.
4. Enter valid information into every field.

Invalid information should display appropriate validation messages and must not create a student record.

Screenshot:

- Required-field validation.
- Invalid email validation.
- Duplicate Student ID validation.
- Validation logic in your code.



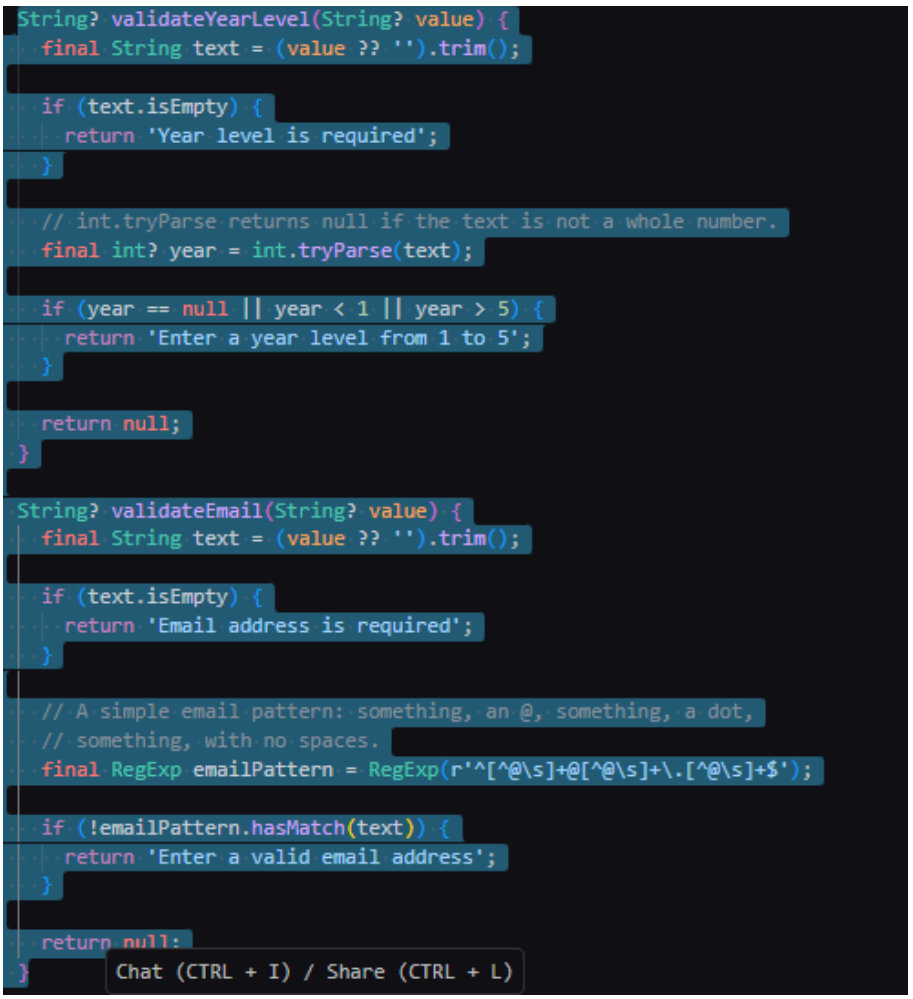

```dart
String? validateStudentId(String? value) {
  final String text = (value ?? '').trim();

  if (text.isEmpty) {
    return 'Student ID is required';
  }

  // Duplicate check: compare with every existing student's ID,
  // ignoring capital letters and extra spaces.
  for (final student in allStudents) {
    if (student.studentId.trim().toLowerCase() == text.toLowerCase()) {
      return 'This Student ID already exists';
    }
  }

  return null;
}
```

Chat (CTRL + I) / Share (CTRL + L)



```dart
String? validateYearLevel(String? value) {
  final String text = (value ?? '').trim();

  if (text.isEmpty) {
    return 'Year level is required';
  }

  // int.tryParse returns null if the text is not a whole number.
  final int? year = int.tryParse(text);

  if (year == null || year < 1 || year > 5) {
    return 'Enter a year level from 1 to 5';
  }

  return null;
}

String? validateEmail(String? value) {
  final String text = (value ?? '').trim();

  if (text.isEmpty) {
    return 'Email address is required';
  }

  // A simple email pattern: something, an @, something, a dot,
  // something, with no spaces.
  final RegExp emailPattern = RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$');

  if (!emailPattern.hasMatch(text)) {
    return 'Enter a valid email address';
  }

  return null;
}
```


🏴 Flag 3 — Enroll and Display a New Student

Objective: Add a successfully enrolled student to your existing Student Directory.

When valid information is submitted:

- Create a new student record.
- Add the student to your existing student list.
- Return to the Student List Screen or update it appropriately.
- Display the newly enrolled student.
- Clear the form fields after successful submission.

Research how Flutter handles form submission, updating application state, and returning from a screen using navigation.

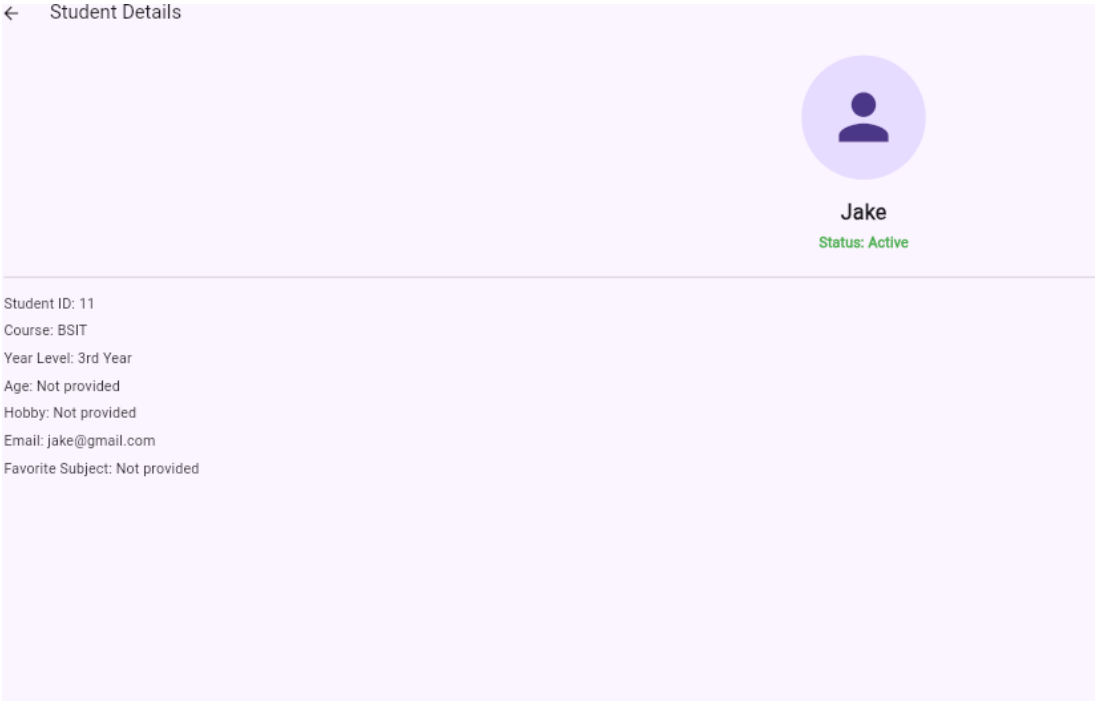
Test

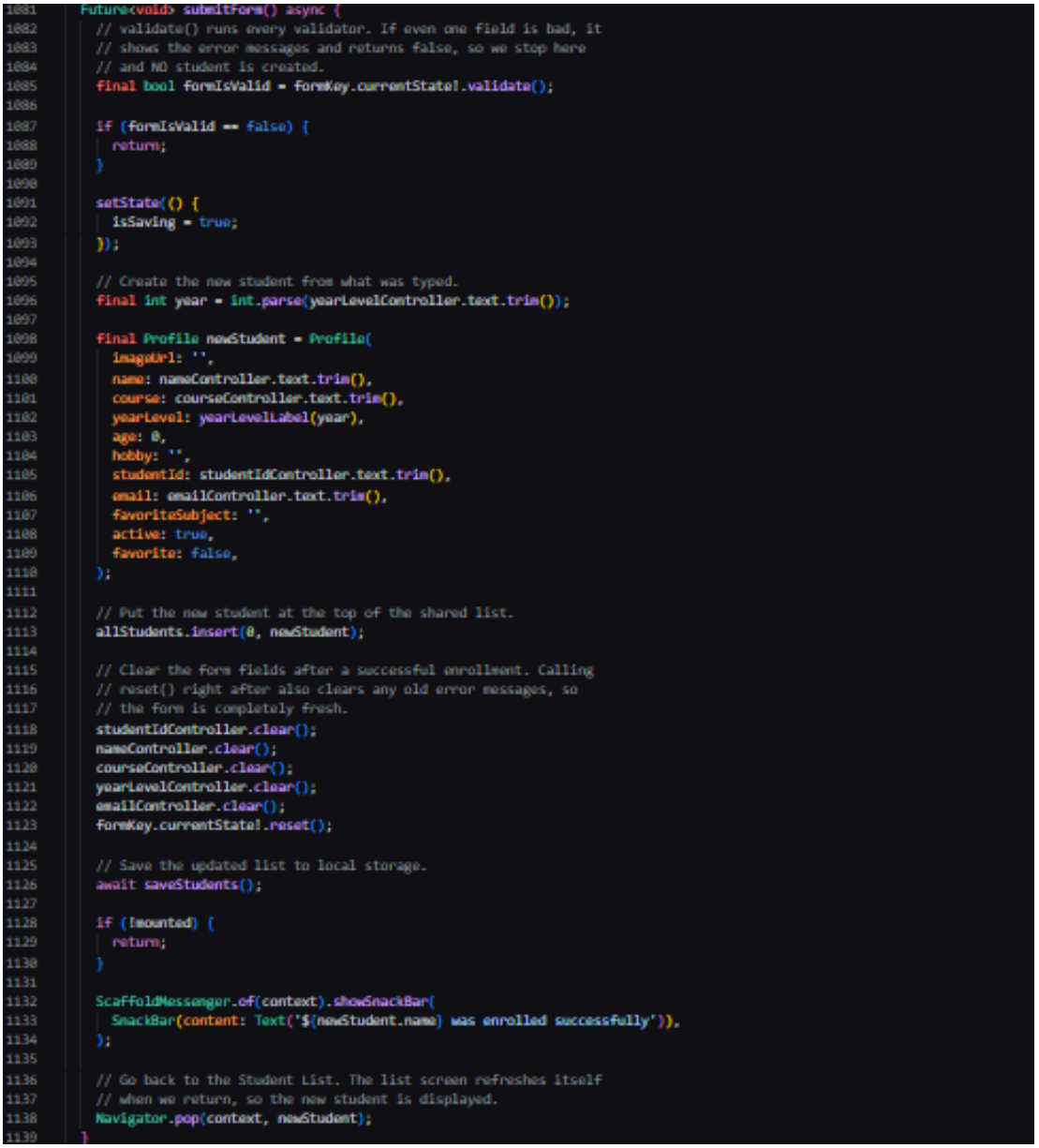
1. Open the Add Student Screen.

2. Enter valid student information.
3. Submit the form.
4. Verify that the new student appears in the Student List.
5. Select the new student.
6. Verify that the Student Details Screen displays the correct information.

Screenshot:

- Completed enrollment form.
- Newly enrolled student in the Student List.
- Student Details Screen displaying the new student's information.
- Submission and navigation logic.





```dart
Future<void> submitForm() async {
  // validate() runs every validator. If even one field is bad, it
  // shows the error messages and returns false, so we stop here
  // and NO student is created.
  final bool formIsValid = formKey.currentState!.validate();

  if (formIsValid == false) {
    return;
  }

  setState(() {
    isSaving = true;
  });

  // Create the new student from what was typed.
  final int year = int.parse(yearLevelController.text.trim());

  final Profile newStudent = Profile(
    imageUrl: '',
    name: nameController.text.trim(),
    course: courseController.text.trim(),
    yearLevel: yearLevelLabel(year),
    age: 0,
    hobby: '',
    studentId: studentIdController.text.trim(),
    email: emailController.text.trim(),
    favoriteSubject: '',
    active: true,
    favorite: false,
  );

  // Put the new student at the top of the shared list.
  allStudents.insert(0, newStudent);

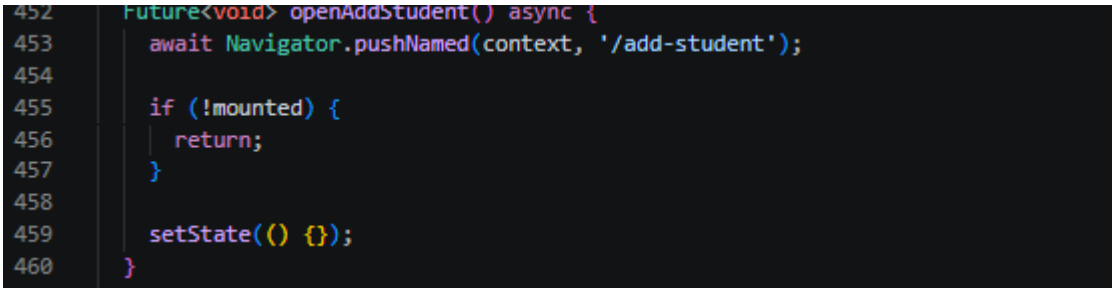
  // Clear the form fields after a successful enrollment. Calling
  // reset() right after also clears any old error messages, so
  // the form is completely fresh.
  studentIdController.clear();
  nameController.clear();
  courseController.clear();
  yearLevelController.clear();
  emailController.clear();
  formKey.currentState!.reset();

  // Save the updated list to local storage.
  await saveStudents();

  if (!mounted) {
    return;
  }

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text('${newStudent.name} was enrolled successfully')),
  );

  // Go back to the Student List. The list screen refreshes itself
  // when we return, so the new student is displayed.
  Navigator.pop(context, newStudent);
}
```



```dart
Future<void> openAddStudent() async {
  await Navigator.pushNamed(context, '/add-student');

  if (!mounted) {
    return;
  }

  setState(() {});
}
```

🏴 Flag 4 — Save Student Records Locally

Objective: Preserve student records even after the application is closed and restarted.

Research Flutter-compatible local data persistence options and choose an appropriate method.

Your implementation must:

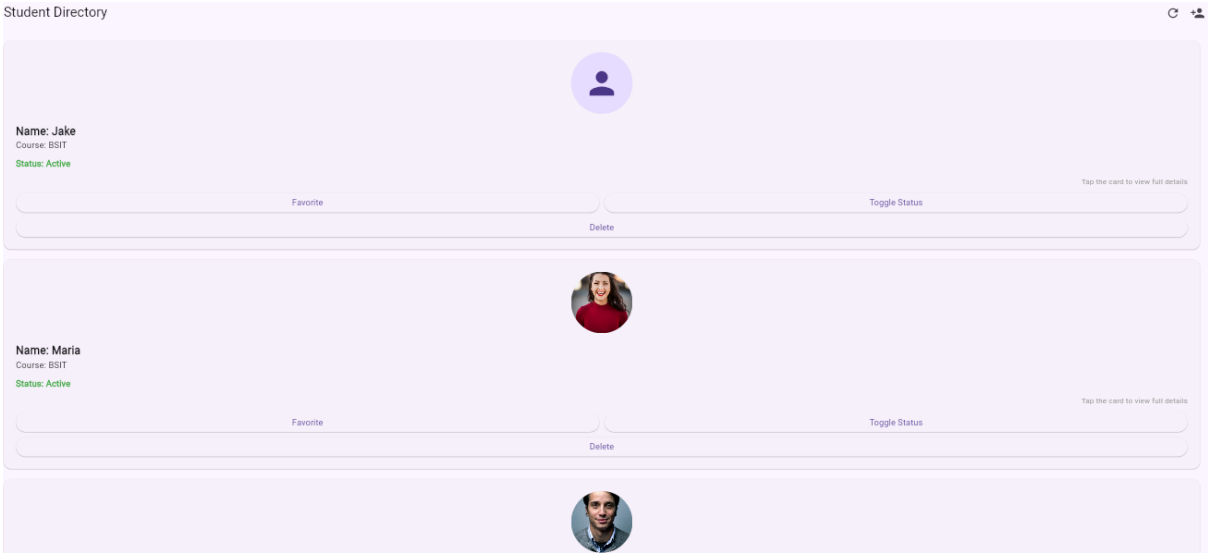
- Save student records locally.
- Load saved records when the application starts.
- Preserve newly enrolled students after restarting the application.
- Handle situations where no saved records exist yet.

Test

1. Enroll a new student.
2. Close and restart the application.
3. Verify that the newly enrolled student still appears in the Student List.
4. Open the student's details and verify that the information remains correct.

Screenshot:

- Student List displaying saved records.
- Local persistence implementation.
- Student Details Screen displaying a saved student after restarting the application.



```dart
// SAVE: turns the whole list into JSON text and stores it locally.
// Every time a student is added, edited, or deleted, we call this
// so the saved data always matches what is on the screen.
Future<void> saveStudents() async {
  final prefs = await SharedPreferences.getInstance();

  final List<Map<String, dynamic>> data = [];
  for (final student in allStudents) {
    data.add(student.toMap());
  }

  await prefs.setString(storageKey, jsonEncode(data));
}

// LOAD: reads the saved JSON text and rebuilds the student list.
Future<void> loadSavedStudents() async {
  final prefs = await SharedPreferences.getInstance();
  final String? savedText = prefs.getString(storageKey);

  // Nothing has been saved yet (first time the app is opened), so we
  // start with the sample students and save them.
  if (savedText == null) {
    allStudents = createSampleStudents();
    await saveStudents();
    return;
  }

  try {
    final List<dynamic> decoded = jsonDecode(savedText) as List<dynamic>;
    final List<Profile> loaded = [];

    for (final item in decoded) {
      loaded.add(Profile.fromMap(item as Map<String, dynamic>));
    }

    // If the saved list is empty (everyone was deleted), it stays
    // empty. Deleted students do NOT come back after a restart.
    allStudents = loaded;
  } catch (_) {
    // The saved data was damaged, so we fall back to the samples
    // instead of crashing the app.
    allStudents = createSampleStudents();
    await saveStudents();
  }
```

```dart
  WidgetsFlutterBinding.ensureInitialized();
  await loadSavedStudents();
  runApp(const MyApp());
}
```

🏴 Flag 5 — Keep Existing Features Functional

Objective: Ensure that enrollment works with the navigation and interactions implemented in CTF 7.

Your application must retain:

- Student List Screen.
- Student Details Screen.

- Student information passed between screens.
- Edit modal.
- Add Student Screen.
- Existing named routes.

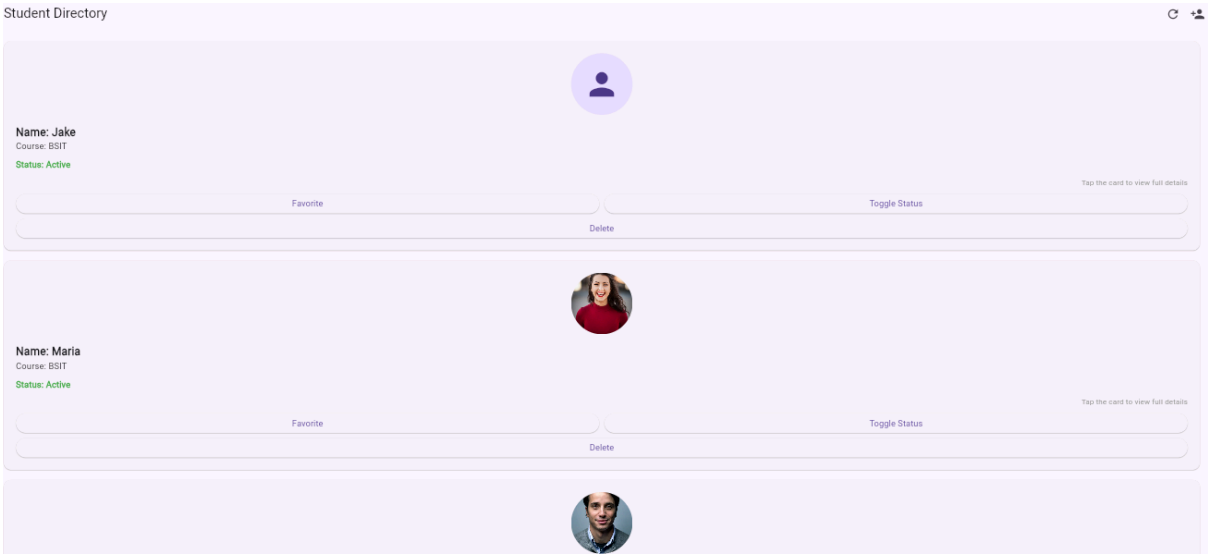
The enrollment workflow must integrate with these existing features rather than replacing them with a single screen.

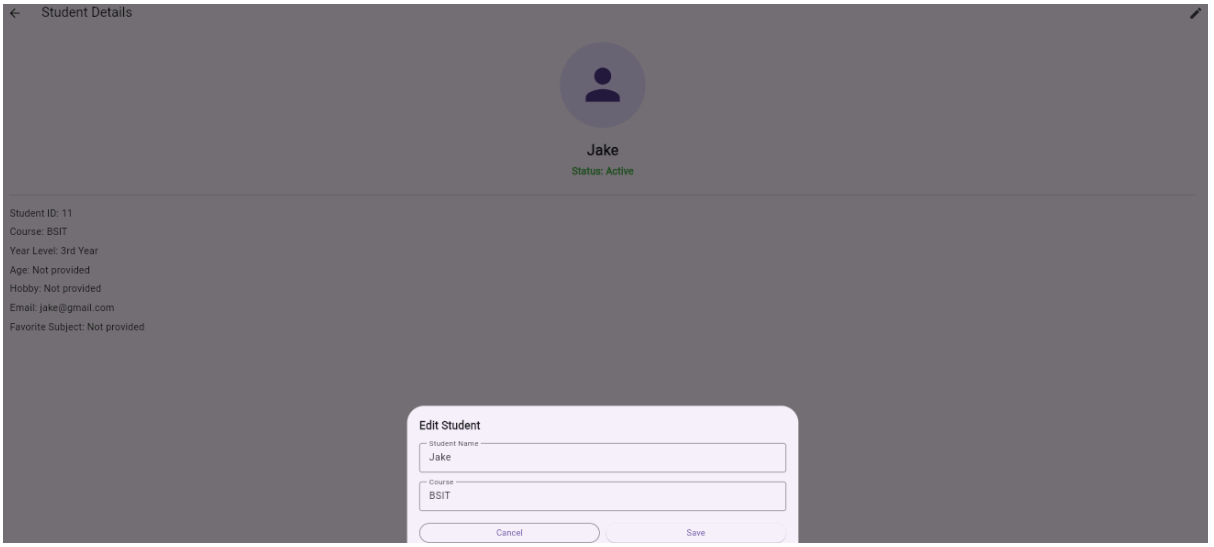
Test

1. Navigate from Student List to Add Student.
2. Enroll a new student.
3. Open the newly enrolled student's details.
4. Open the Edit modal.
5. Modify the student's information.
6. Close the modal and verify the updated information.
7. Navigate back to the Student List.
8. Verify that the student information remains consistent.

Screenshot:

- Student List with the newly enrolled student.
- Student Details displaying the correct information.
- Edit modal functioning correctly.
- Existing navigation working with the enrollment feature.






```
289       initialRoute: '/student-list',
290
291       // These are the simple routes. A simple route does not need
292       // any extra information to open, it just opens the screen.
293       routes: {
294         '/student-list': (context) => const StudentListScreen(),
295         '/add-student': (context) => const AddStudentScreen(),
296       },
297
298       // The Student Details screen is a bit different because it
299       // needs to know WHICH student was tapped before it can open.
300       // Because of that, we cannot put it in the simple routes list
301       // above. Instead we catch it here and manually read the
302       // student that was passed in.
303       onGenerateRoute: (settings) {
304         if (settings.name == '/student-details') {
305           // settings.arguments holds whatever data was sent when
306           // this route was opened. We expect it to be a Profile.
307           final selectedStudent = settings.arguments as Profile;
308
309           return MaterialPageRoute(
310             builder: (context) {
311               return StudentDetailsScreen(student: selectedStudent);
312             },
313           ); // MaterialPageRoute
314         }
315
316         // If the route name does not match anything we know,
317         // we return null and Flutter will handle the error itself.
318         return null;
319       },
```

🏴 Flag 6 — Synchronize Changes with Local Data

Objective: Ensure that creating, editing, and deleting students updates the locally saved records.

Your application should keep its student list and locally persisted data consistent.

Verify that:

- Newly enrolled students are saved.
- Edited student information is saved.
- Deleted student records remain deleted after restarting.
- Student IDs remain unique.
- The Student List reflects the latest student records.

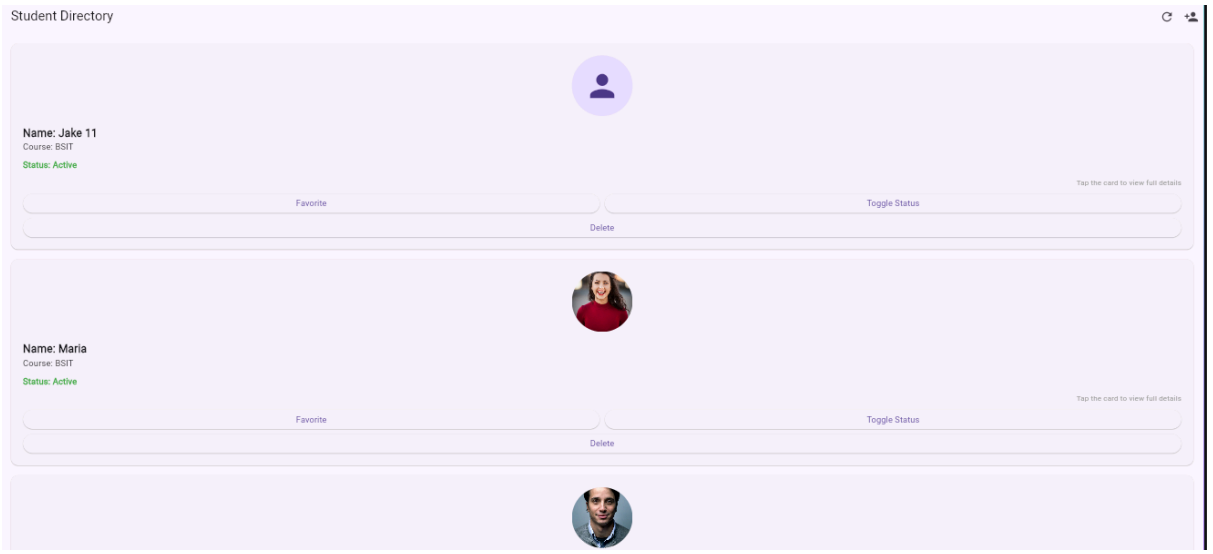
Research how to update and remove persisted records using your chosen local storage method.

Test

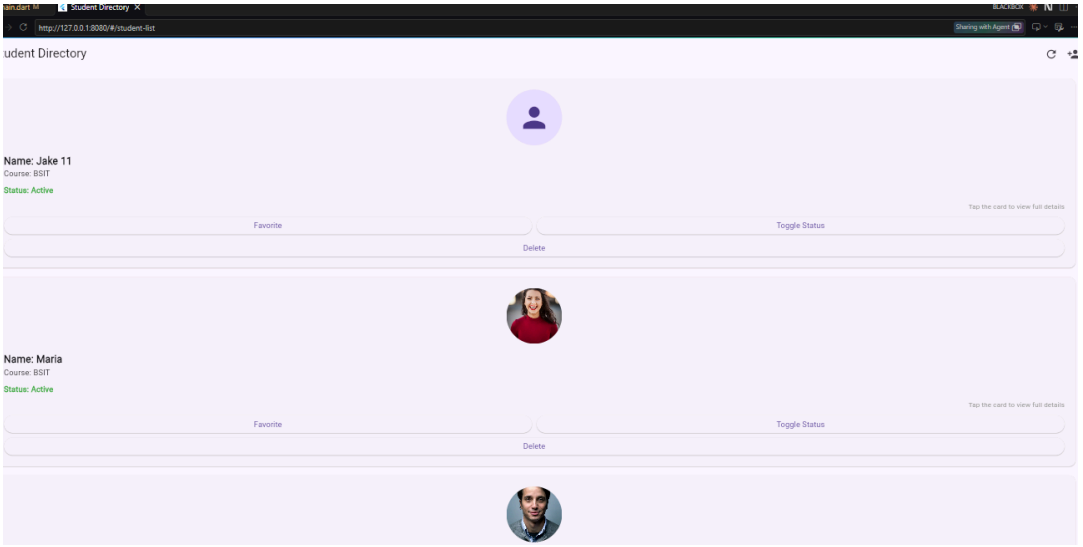
1. Enroll a new student.
2. Edit the student's information using the modal.
3. Delete a student using your existing delete functionality, if implemented.
4. Restart the application.
5. Verify that the saved records reflect the changes.

Screenshot:

- Student List after enrollment and editing.
- Updated student details.
- Student records after restarting the application.
- Persistence logic for creating, updating, and deleting records.








```
                Expanded(
                  child: ElevatedButton(
                    onPressed: () {
                      setState(() {
                        widget.student.name = nameController.text;
                        widget.student.course = courseController.text;
                      });

                      // Save the edited student to local storage
                      // so the change survives a restart.
                      saveStudents();

                      // Closes the modal after saving.
                      Navigator.pop(context);
                    },
                    child: const Text('Save'),
                  ), // ElevatedButton
                ), // Expanded
```

```dart
// This removes a student from the list completely.
void deleteStudent(int index) {
  setState(() {
    students.removeAt(index);
  });

  // Save the shorter list so the deleted student stays deleted
  // after a restart.
  saveStudents();
}
```

```dart
// Put the new student at the top of the shared list.
allStudents.insert(0, newStudent);

// Clear the form fields after a successful enrollment. Calling
// reset() right after also clears any old error messages, so
// the form is completely fresh.
studentIdController.clear();
nameController.clear();
courseController.clear();
yearLevelController.clear();
emailController.clear();
formKey.currentState!.reset();

// Save the updated list to local storage.
await saveStudents();
```

🏴 Flag 7 — Verify the Complete Enrollment Workflow

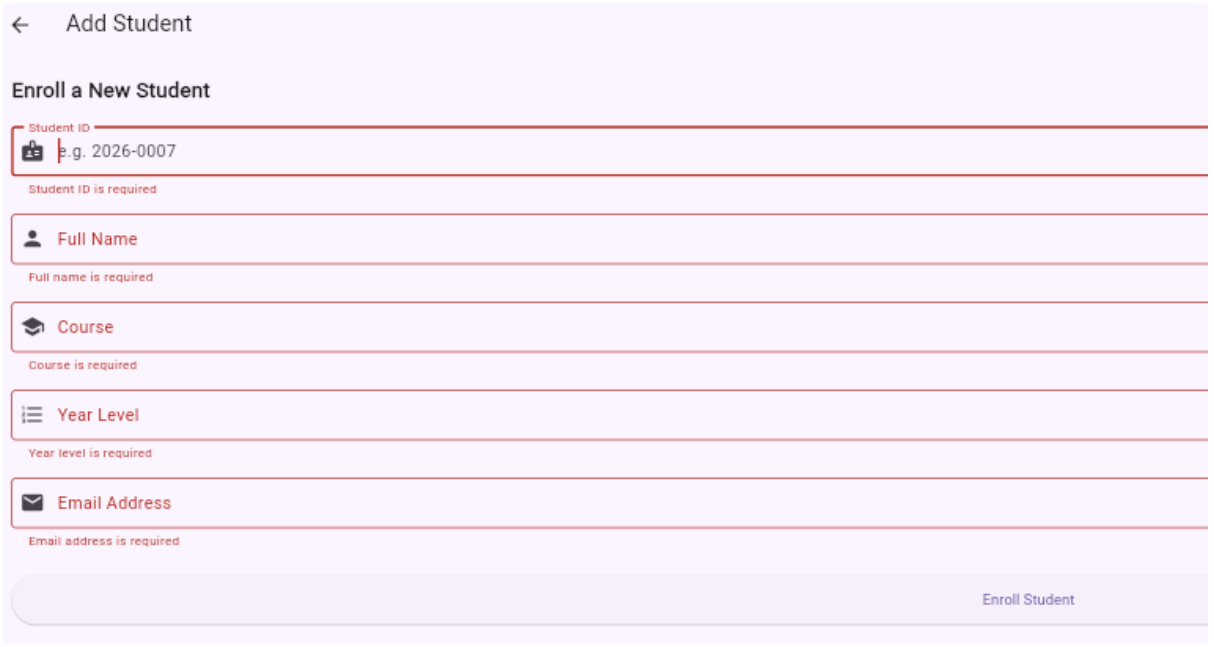
Objective: Demonstrate that the Student Directory supports enrollment, validation, navigation, editing, and local data persistence.

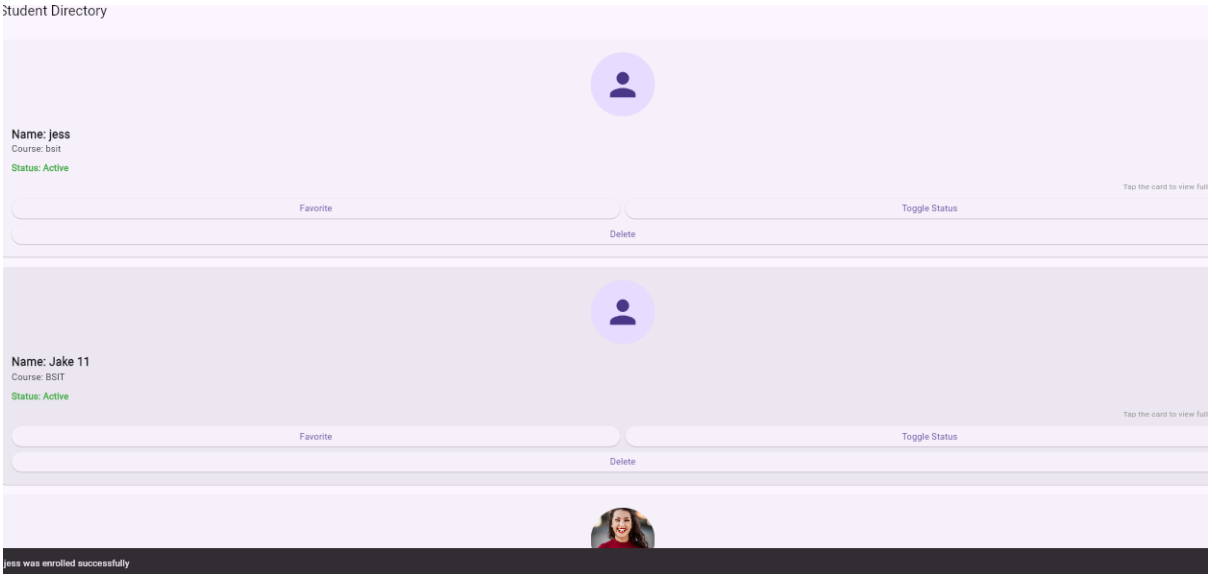
Test Scenario

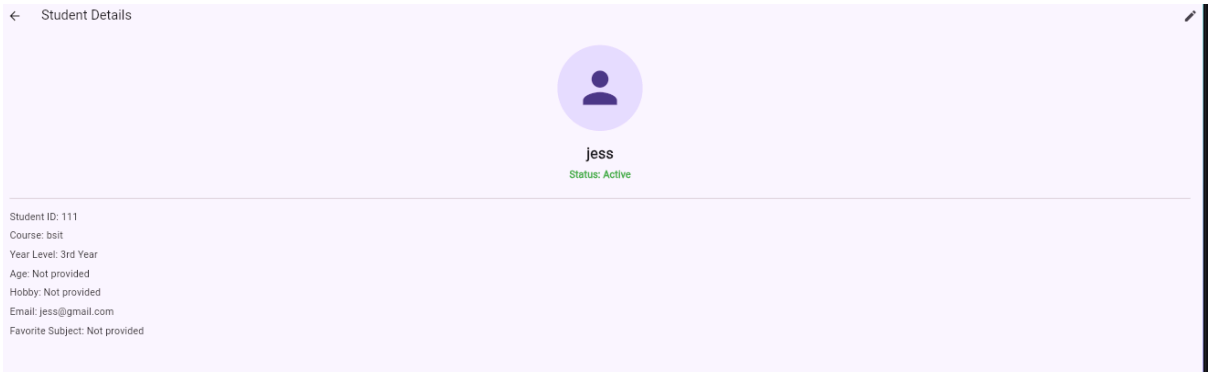
1. Launch the application.
2. Verify that saved student records load.
3. Navigate to Add Student.
4. Submit an empty form.
5. Submit an invalid email address.
6. Attempt to enroll a student with a duplicate Student ID.
7. Enter valid student information.
8. Submit the form successfully.
9. Verify that the student appears in the Student List.
10. Open Student Details and verify the student's information.
11. Edit the student using the modal.
12. Restart the application.
13. Verify that the enrolled student and updated information remain available.

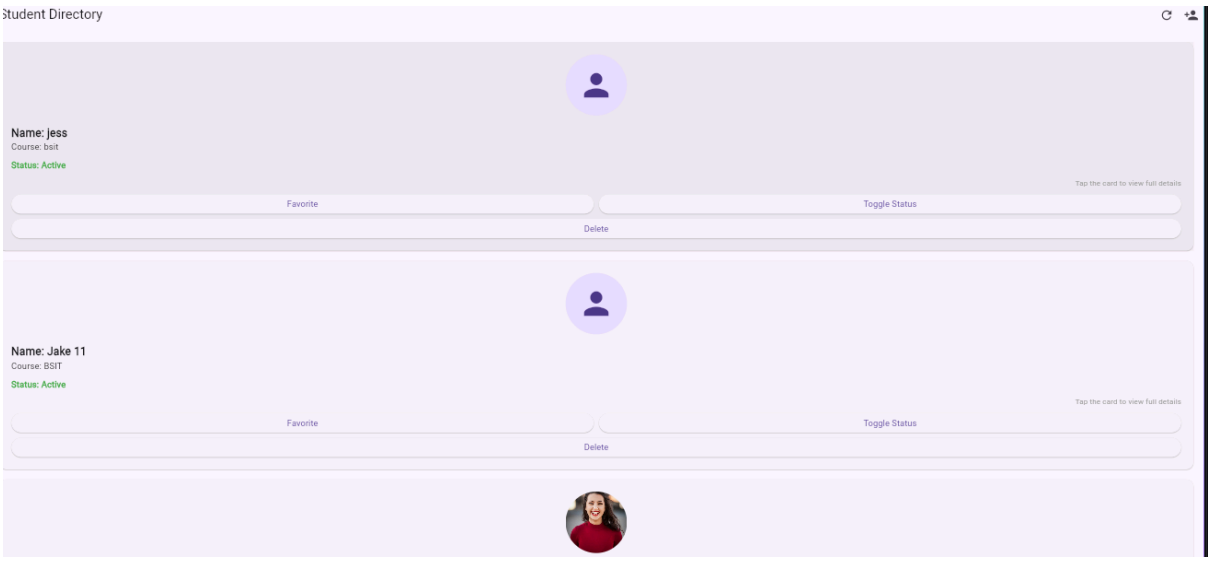
Screenshot:

- Enrollment form with validation errors.
- Successful enrollment.
- Student List displaying the new student.
- Student Details and Edit modal.
- Saved student information after restarting the application.


Add Student
Enroll a New Student
Student ID
e.g. 2026-0007
Student ID is required
Full Name
Full name is required
Course
Course is required
Year Level
Year level is required
Email Address
Email address is required
Enroll Student


Student Directory
Name: jess
Course: bsit
Status: Active
Favorite
Toggle Status
Delete
Name: Jake 11
Course: BSIT
Status: Active
Favorite
Toggle Status
Delete
jess was enrolled successfully


Student Details
jess
Status: Active
Student ID: 111
Course: bsit
Year Level: 3rd Year
Age: Not provided
Hobby: Not provided
Email: jess@gmail.com
Favorite Subject: Not provided



🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student Enrollment feature.

Use a proper Git commit message.

Branch:

`feature/student-enrollment`

Screenshot:

- Successful commit.
- Current branch.
- Terminal showing the commit.
- Successful push to the remote repository.

```
PS C:\Users\Lenovo\OneDrive\Desktop\feature-switch> git status --short --branch
## feature/student-enrollment...origin/feature/student-enrollment
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\feature-switch> git commit --allow-empty -m "feat: finalize student enrollment
feature"
[feature/student-enrollment c22a68c] feat: finalize student enrollment feature
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\feature-switch> git push -u origin feature/student-enrollment
To https://github.com/JP3756/IT314AB_Cabaluna.git
 * [new branch]      feature/student-enrollment -> feature/student-enrollment
branch 'feature/student-enrollment' set up to track 'origin/feature/student-enrollment'.
```

```
PS C:\Users\Lenovo\OneDrive\Desktop\feature-switch> git log -1 --oneline
c22a68c feat: finalize student enrollment feature
```